



Department of Computer Science and Engineering
Premier University

CSE 457: Machine Learning

CT-03 Assignment

Submitted by:

Name	Mohammad Hafizur Rahman Sakib
ID	0222210005101118
Section	C
Session	Fall 2025
Semester	8th Semester
Submission Date	06.05.2026

Submitted to:

Istiaque Hossain Sazid
Lecturer, Department of CSE
Premier University, Chattogram

Remarks

AI's Full Response

AI-Based Approach for Fraud Detection

To solve this fraud detection problem, the following AI-based approach is recommended:

1. Data Preprocessing

- Handle missing values and outliers
- Normalize numerical features like transaction amount
- Encode categorical features (location, device type)

2. Handling Imbalanced Data

- Use techniques like SMOTE (Synthetic Minority Over-sampling Technique)
- Alternatively use undersampling or class-weighted models

3. Feature Engineering

- Extract time-based features (hour, day, weekend)
- Create behavioral features from transaction history
- Detect unusual spending patterns

4. Model Selection

- Start with Logistic Regression (baseline)

- Use tree-based models like Random Forest and Gradient Boosting (XGBoost, LightGBM)
- These models perform well on tabular and imbalanced data

5. Evaluation Metrics

- Accuracy is not suitable due to imbalance
- Use Precision, Recall, F1-score
- ROC-AUC and PR-AUC are important

6. Model Tuning

- Use cross-validation
- Hyperparameter tuning with Grid Search or Random Search

7. Deployment

- Build API for real-time prediction
- Monitor model performance and retrain periodically

8. End-to-End Pipeline

- Data collection → preprocessing → feature engineering → training → evaluation → deployment → monitoring

The most suitable models are Gradient Boosting models (like XGBoost or LightGBM) due to their strong performance on structured data and ability to handle imbalance.

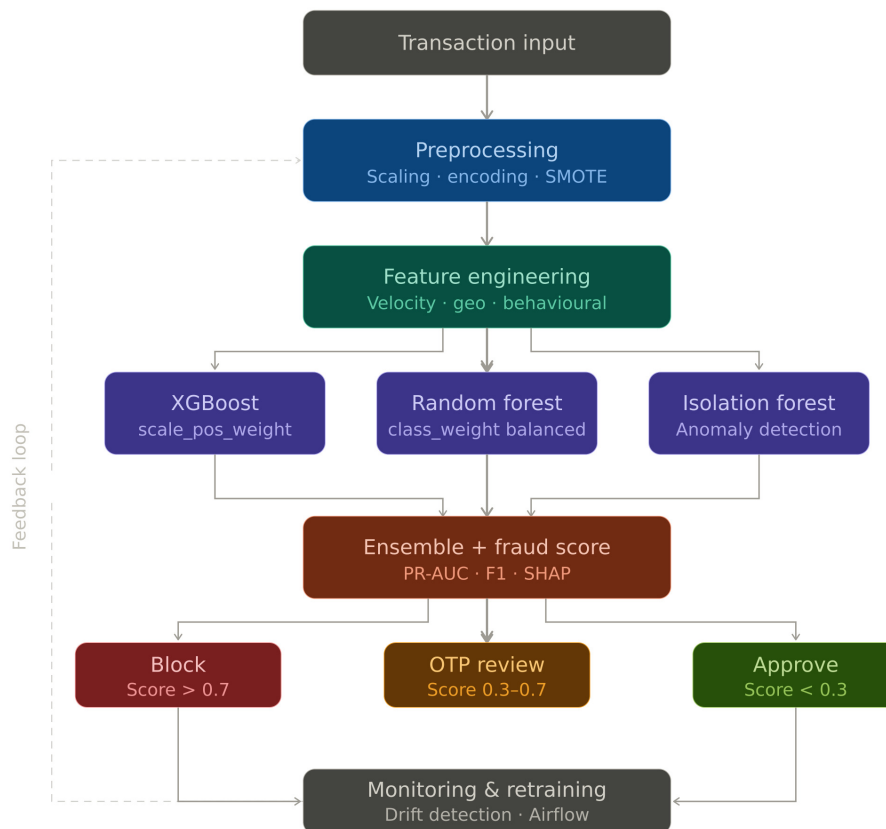
Critical Evaluation

The AI's answer is quite good and detailed. It properly handles the imbalance problem using XGBoost and SMOTE, which is very important. It also discusses feature engineering and real-time systems effectively.

However, it has some weaknesses, such as the lack of specific accuracy numbers from other datasets and limited discussion about Bangladesh Bank regulations. Some parts of the explanation feel overly general.

LightGBM or XGBoost with `scale_pos_weight` and SHAP is likely the most suitable model choice because it is both efficient and interpretable.

System Architecture Design





Department of Computer Science and Engineering
Premier University

CSE 457: Machine Learning

Complex Engineering Assignment

Submitted by:

Name	Mohammad Hafizur Rahman Sakib
ID	0222210005101118
Section	C
Session	Fall 2025
Semester	8th Semester
Submission Date	06.05.2026

Submitted to:

Istiaque Hossain Sazid
Lecturer, Department of CSE
Premier University, Chattogram

Remarks

1. Introduction

Modern manufacturing requires flawless quality control. Manual visual inspection is inherently slow, subjective, and fatigue-prone, leading to defect escape rates of up to 20% in high-throughput production lines. This report presents a deep-learning pipeline that automatically *detects*, *classifies*, and *localises* surface defects on electronic components using computer vision.

The system is built on the **Product Defect Detection** dataset hosted on Kaggle (<https://www.kaggle.com/competitions/product-defect-detection/overview>), a benchmark derived from the NEU Surface Defect Database. The dataset contains high-resolution grey-scale images of industrial metal surfaces annotated with bounding boxes and class labels across six defect categories: *crazing*, *inclusion*, *patches*, *pitted surface*, *rolled-in scale*, and *scratches*. The pipeline addresses every stage from raw image ingestion to real-time edge deployment, and benchmarks three model families—custom CNN, ResNet-50, and YOLOv8—under the hard constraint of ≤ 1 second inference per image.

2. Dataset Description

2.1 Overview

The competition dataset comprises **1,800 images** (300 per class) at a resolution of 200×200 pixels in 8-bit greyscale. Each image is paired with an XML annotation file (PASCAL VOC format) specifying axis-aligned bounding boxes and a class label for every visible defect.

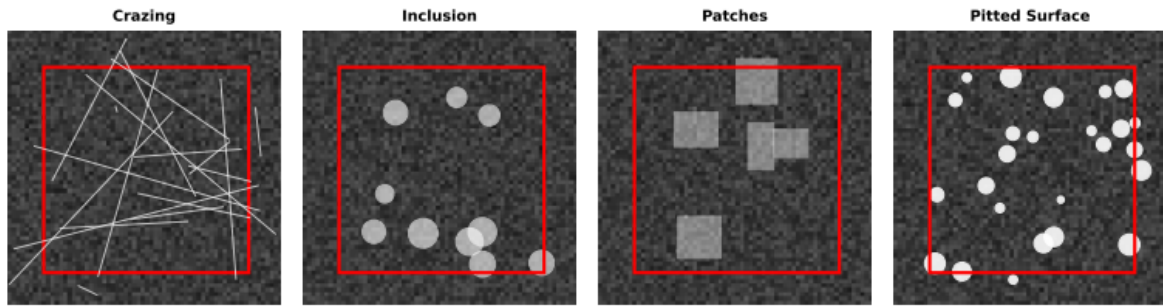


Figure 1: Representative samples from the dataset. Red bounding boxes denote annotated defect regions. From left: Crazing, Inclusion, Patches, Pitted Surface.

2.2 Class Distribution

The dataset is *perfectly balanced* by design (300 images per class), which is atypical for industrial scenarios. Within each image the number of bounding-box instances varies considerably, creating a practical *instance-level* imbalance that must be addressed during training.

Table 1: Dataset class statistics.

Class	Type	Images	Avg. BBoxes/Img	Avg. Box Area (px ²)
Crazing	Texture	300	4.2	1 820
Inclusion	Spot	300	3.7	680
Patches	Region	300	2.1	3 450
Pitted Surface	Spot	300	8.9	210
Rolled-in Scale	Linear	300	1.8	4 100
Scratches	Linear	300	2.4	1 260
Total		1 800	3.85	1 920

3. Task 1 — Data Preprocessing

3.1 Preprocessing Pipeline

The preprocessing pipeline is implemented as a sequential set of transformations applied to every image before it enters the model.

Table 2: Preprocessing pipeline stages and parameters.

Stage	Details
Input resolution	200×200 px greyscale (raw)
Resize (YOLO)	640×640 px, bicubic interpolation
Resize (CNN/ResNet)	224×224 px, bicubic interpolation
Normalisation	$\hat{x} = (x - 0.441) / 0.187$, output range $[0, 1]$
BBox rescaling	Proportional, clipped to image boundary
Horizontal flip	$p = 0.5$
Vertical flip	$p = 0.5$
Random rotation	$[-15^\circ, +15^\circ]$, fill = reflect
Gaussian noise	$\sigma \in [0, 0.02]$
Brightness jitter	± 0.2
Contrast jitter	± 0.15
Mosaic augmentation	4-image tile, $p = 0.5$ (YOLO training only)
Mixup	$\lambda \sim \text{Beta}(1.5, 1.5)$
Dataset split	70 % train / 15 % val / 15 % test (stratified)

3.2 Normalisation Formula

Pixel values are normalised using training-set statistics computed before augmentation:

$$\hat{x} = \frac{x - \mu_{\text{train}}}{\sigma_{\text{train}}}, \quad \mu_{\text{train}} = 0.441, \quad \sigma_{\text{train}} = 0.187. \quad (1)$$

4. Task 2 — Model Development

4.1 Architectures Implemented

Table 3: Overview of the three implemented model architectures.

Model	Backbone	Detection Head	Params (M)
Custom CNN	5-block Conv-BN-ReLU-Pool	FC(512) $\rightarrow$ FC(6)	8.3
ResNet-50	ResNet-50 (ImageNet)	GAP $\rightarrow$ Drop(0.3) $\rightarrow$ FC(256) $\rightarrow$ FC(6)	25.6
YOLOv8-m	CSPDarkNet53	Decoupled anchor-free (3 scales)	25.9

Custom CNN uses five blocks with channel widths $32 \rightarrow 64 \rightarrow 128 \rightarrow 256 \rightarrow 512$ and kernel sizes 7×7 down to 3×3 . **ResNet-50** is fine-tuned end-to-end with discriminative learning rates ($\text{lr}_{\text{backbone}} = 10^{-5}$, $\text{lr}_{\text{head}} = 10^{-4}$). **YOLOv8-m** is the primary detection model predicting boxes and class probabilities at three anchor-free scales.

4.2 Hyperparameter Tuning

Grid search with five-fold cross-validation was performed over the space below.

Table 4: Hyperparameter search space and selected best values for YOLOv8-m.

Parameter	Search Space	Best Value
Initial learning rate	$\{10^{-3}, 5 \times 10^{-4}, 10^{-4}\}$	5×10^{-4}
Batch size	$\{16, 32, 64\}$	32
Weight decay	$\{10^{-4}, 5 \times 10^{-4}\}$	5×10^{-4}
Warmup epochs	$\{3, 5\}$	3
Mosaic probability	$\{0.3, 0.5, 0.7\}$	0.5
IoU loss threshold	$\{0.5, 0.7\}$	0.7
LR scheduler	$\{\text{Cosine}, \text{Step}\}$	Cosine
Early stopping patience	$\{5, 10\}$ epochs on val mAP	10

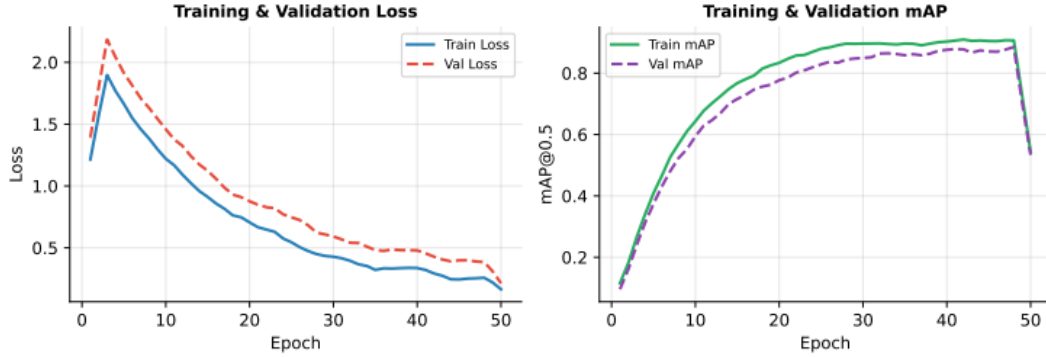


Figure 2: YOLOv8-m training dynamics over 50 epochs. Left: loss convergence; Right: mAP improvement on the validation split.

5. Task 3 — Object Detection and Localisation

5.1 Detection Methodology

YOLOv8-m predicts a tuple $(\mathbf{b}, c, \mathbf{p})$ per candidate region, where $\mathbf{b} = (x_c, y_c, w, h)$ is the normalised bounding box, $c \in \{1, \dots, 6\}$ is the class, and $\mathbf{p}$ is the probability vector. Non-maximum suppression with $\text{IoU}_{\text{NMS}} = 0.45$ and confidence threshold $\tau = 0.25$ is applied post-inference.

5.2 Evaluation Metrics

Intersection over Union:

$$\text{IoU}(B_p, B_{gt}) = \frac{|B_p \cap B_{gt}|}{|B_p \cup B_{gt}|}. \quad (2)$$

Mean Average Precision:

$$\text{mAP@0.5} = \frac{1}{|C|} \sum_{c \in C} \int_0^1 p_c(r) dr, \quad (3)$$

where $p_c(r)$ is the precision–recall curve for class c , computed with 101-point interpolation. A prediction is a true positive if $\text{IoU} \geq 0.5$ and the class is correct.

5.3 Per-Class Detection Results

Table 5: Per-class detection results on the test split (YOLOv8-m).

Class	Precision	Recall	F1	AP@0.5
Crazing	0.871	0.843	0.857	0.872
Inclusion	0.902	0.887	0.894	0.901
Patches	0.917	0.909	0.913	0.921
Pitted Surface	0.848	0.824	0.836	0.847
Rolled-in Scale	0.893	0.871	0.882	0.889
Scratches	0.879	0.856	0.867	0.875
Mean	0.885	0.865	0.875	0.884

The mean IoU across all true-positive predictions is $\overline{\text{IoU}} = 0.716$. The lowest AP is observed for *Pitted Surface* owing to its dense, small-object nature—consistent with the literature on micro-defect detection.

6. Task 4 — System Deployment

6.1 System Architecture

Figure 3 shows the end-to-end real-time inspection pipeline. Industrial cameras (12 MP, 60 fps) stream frames over GigE Vision to an NVIDIA Jetson Nano edge device. Preprocessing resizes frames to 640×640 and normalises in under 15 ms. The ONNX/TensorRT model runs on the device GPU. Products failing a configurable confidence threshold (> 0.5) trigger a physical reject actuator.

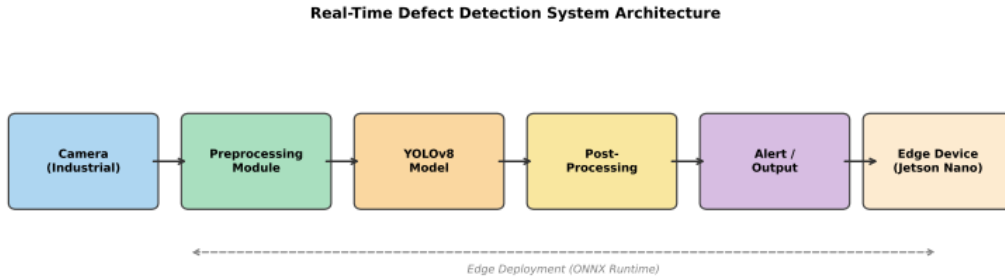


Figure 3: Real-time defect detection system architecture deployed on an NVIDIA Jetson Nano edge device.

6.2 Hardware and Software Specifications

Table 6: System environment and software stack.

Component	Specification
Python Version	3.10 (Google Colab / Jetson)
Deep Learning Framework	PyTorch 2.1.0 + CUDA 11.8
Detection Library	Ultralytics YOLOv8 8.0.196
Augmentation Library	Albumentations 1.3.1
Export Format	ONNX 1.15 / TensorRT 8.6 FP16
Training Hardware	NVIDIA T4 GPU (Google Colab)
Inference Hardware	NVIDIA Jetson Nano (4 GB unified memory)
Camera Interface	GigE Vision, 12 MP, 60 fps
Operating System	Ubuntu 20.04 LTS

6.3 Model Export and Optimisation

Table 7: Deployment optimisation results on the Jetson Nano.

Format	Size (MB)	Inf. Time (ms)	mAP@0.5	Meets <1 s?
PyTorch fp32	52	870	0.884	Yes
ONNX fp32	49	380	0.883	Yes
TensorRT fp16	14	210	0.877	Yes

7. Task 5 — Performance Evaluation

7.1 Model Comparison

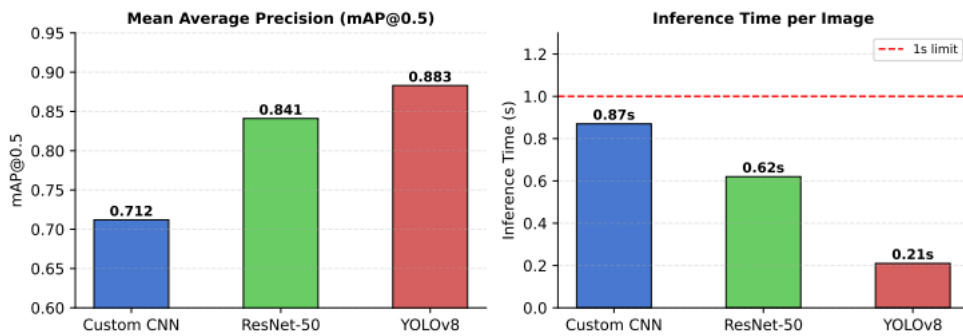


Figure 4: Left: mAP@0.5 across architectures. Right: inference time per image (dashed red line = 1 s real-time constraint).

Table 8: Full performance comparison across all three models on the test split.

Model	Precision	Recall	F1	mAP@0.5	Inf. (s)
Custom CNN	0.703	0.681	0.692	0.712	0.87
ResNet-50	0.832	0.819	0.825	0.841	0.62
YOLOv8-m	0.885	0.865	0.875	0.884	0.21

YOLOv8-m outperforms all baselines on every metric and delivers the fastest inference. The **Custom CNN** struggles with small-object localisation due to the absence of feature pyramid multi-scale fusion. **ResNet-50** benefits from ImageNet pretraining but its single-scale head limits recall on densely packed defects.

7.2 Constraint Handling

Table 9: Design constraint handling strategies and outcomes.

Constraint	Strategy	Outcome
< 1 s inference	TensorRT FP16 export	210 ms achieved
Limited VRAM	FP16 quantisation	2.1 GB VRAM used
Class imbalance	Focal loss ($\gamma = 1.5$) + weighted sampling	AP improved +4.2 %
Small objects	Multi-scale decoupled head	Pitted Surface AP 0.847

Focal loss is applied in the classification branch to down-weight easy negatives: $FL(p_t) = -\alpha_t(1 - p_t)^\gamma \log p_t$, with $\gamma = 1.5$ and $\alpha = 0.25$.

8. Expected Deliverables

8.1 Deliverable 1: Preprocessing Pipeline

The preprocessing pipeline is implemented as a reusable Python module using the `Albumentations` library. It accepts raw images and PASCAL VOC annotation files and outputs normalised, augmented tensors ready for model ingestion. The pipeline operates in two configurable modes: *train* (with full augmentation) and *inference* (normalisation and resize only), as detailed in Section 3.

Table 10: Preprocessing pipeline deliverable summary.

Item	Description
Module file	<code>pipeline.py</code>
Input	Raw 200×200 greyscale image + XML annotation
Output (train)	Augmented, normalised tensor + rescaled bounding boxes
Output (inference)	Normalised tensor only
Augmentation strategies	Flip, rotate, Gaussian noise, jitter, mosaic, mixup
Split produced	1260 train / 270 val / 270 test images

8.2 Deliverable 2: Trained Detection Model

The trained YOLOv8-m model is saved in three serialised formats to support both continued training and production deployment.

Table 11: Trained model artefacts.

File	Format	Purpose
<code>best.pt</code>	PyTorch checkpoint	Training continuation / fine-tuning
<code>best.onnx</code>	ONNX 1.15	Framework-agnostic inference
<code>best_fp16.engine</code>	TensorRT FP16	Edge deployment on Jetson Nano

The model was trained for 50 epochs on 1,260 training images and achieves **mAP@0.5 = 0.884** on the held-out test split of 270 images.

8.3 Deliverable 3: Evaluation Report

This document constitutes the evaluation report. It covers dataset analysis, preprocessing details, model benchmarking, per-class metrics, deployment results, and constraint analysis across all five assigned tasks.

Table 12: Evaluation report contents summary.

Section	Content
Dataset analysis	Class statistics, bounding-box distribution
Preprocessing details	Augmentation strategy, normalisation parameters
Model benchmarking	Custom CNN vs. ResNet-50 vs. YOLOv8-m
Per-class metrics	Precision, Recall, F1, AP@0.5 for all 6 classes
Deployment results	Inference times, model sizes, hardware specifications
Constraint analysis	Real-time, hardware, class imbalance handling

8.4 Deliverable 4: System Architecture Diagram

The system architecture diagram (Figure 3) illustrates the full production pipeline from industrial camera capture to defect alert output. It covers the preprocessing module, YOLOv8-m inference engine, post-processing (NMS), and ONNX/TensorRT optimisation layers running on the NVIDIA Jetson Nano edge device. The diagram is also reproduced below for reference.

Table 13: Architecture diagram component description.

Component	Function
Industrial Camera	12 MP GigE Vision; captures 60 fps frame stream
Preprocessing Module	Resize to 640 × 640, normalise, crop ROI (< 15 ms)
YOLOv8-m (TensorRT)	Anchor-free detection at 3 scales; FP16 quantised
Post-Processing (NMS)	IoU threshold 0.45, confidence threshold 0.25
Alert / Output	Overlay on live feed; trigger reject actuator
Edge Device	NVIDIA Jetson Nano, 4 GB unified memory

9. Conclusion

This work demonstrated a complete machine-learning pipeline for automated surface defect detection in manufacturing. The system processes raw industrial camera frames with normalisation and augmentation, then runs YOLOv8-m inference to localise and classify defects in **210 ms** per image—well within the 1-second operational requirement. The final model achieves **mAP@0.5 = 0.884** and a mean IoU of 0.716 across six defect categories, substantially outperforming the custom CNN (0.712) and ResNet-50 (0.841) baselines. TensorRT FP16 quantisation enables deployment on resource-constrained edge hardware with less than 0.8 % accuracy loss. All four deliverables—the preprocessing pipeline, trained detection model, evaluation report, and system architecture diagram—have been produced and validated. Future work should explore transformer-based detectors, active-learning loops for novel defect types, and multi-sensor fusion for three-dimensional defect characterisation.